

1. Objetivo

El objetivo del proyecto fue desarrollar una aplicación web que permitiera consultar información sobre videojuegos utilizando la API RAWG. Además de mostrar la información obtenida desde la API, se buscó aplicar los conceptos vistos durante el curso, como Vue Router, componentes reutilizables, Pinia, persistencia con el navegador y una organización clara del proyecto.

2. Tecnologías

- Vue 3
- Vite
- JavaScript
- Vue Router
- Pinia
- HTML5
- CSS3
- RAWG API
- LocalStorage
- SessionStorage

3. Estructura

El proyecto se inició utilizando la estructura base generada por Vite. A partir de ella se incorporaron carpetas como `views`, `stores`, `services` y `composables` para separar responsabilidades y facilitar el mantenimiento del código.

```
src
├── assets
├── components
├── composables
├── router
├── services
├── stores
└── views
```

4. Decisiones arquitectónicas

Durante el desarrollo del proyecto se tomaron las siguientes decisiones para mejorar la organización y el mantenimiento del código:

- **Separación de la lógica de acceso a datos:** se creó la carpeta `services`, donde se centralizan las funciones encargadas de consumir la API. De esta forma, los componentes no realizan directamente las solicitudes y el código resulta más fácil de mantener.
- **Uso de componentes reutilizables:** la interfaz se dividió en componentes independientes, como las tarjetas de juegos, la barra de navegación y el pie de página. Esto permitió reutilizar código y mantener una estructura más ordenada.

- **Manejo de estado con Pinia:** se utilizó Pinia para centralizar la información compartida de la aplicación, principalmente la lista de juegos favoritos. Esto evitó tener que pasar información entre múltiples componentes mediante props.
- **Separación entre estado local y compartido:** la información que solo utiliza un componente (como campos de búsqueda, indicadores de carga o filtros temporales) se mantiene como estado local, mientras que la información utilizada por varias vistas se almacena en Pinia.
- **Persistencia mediante almacenamiento del navegador:** se utilizó `localStorage` para conservar los juegos favoritos entre sesiones y `sessionStorage` para mantener la sesión simulada del usuario durante la navegación.
- **Organización modular del proyecto:** el código se organizó en carpetas como `components`, `views`, `services`, `stores` y `composables`, con el objetivo de separar responsabilidades y facilitar el mantenimiento de la aplicación.

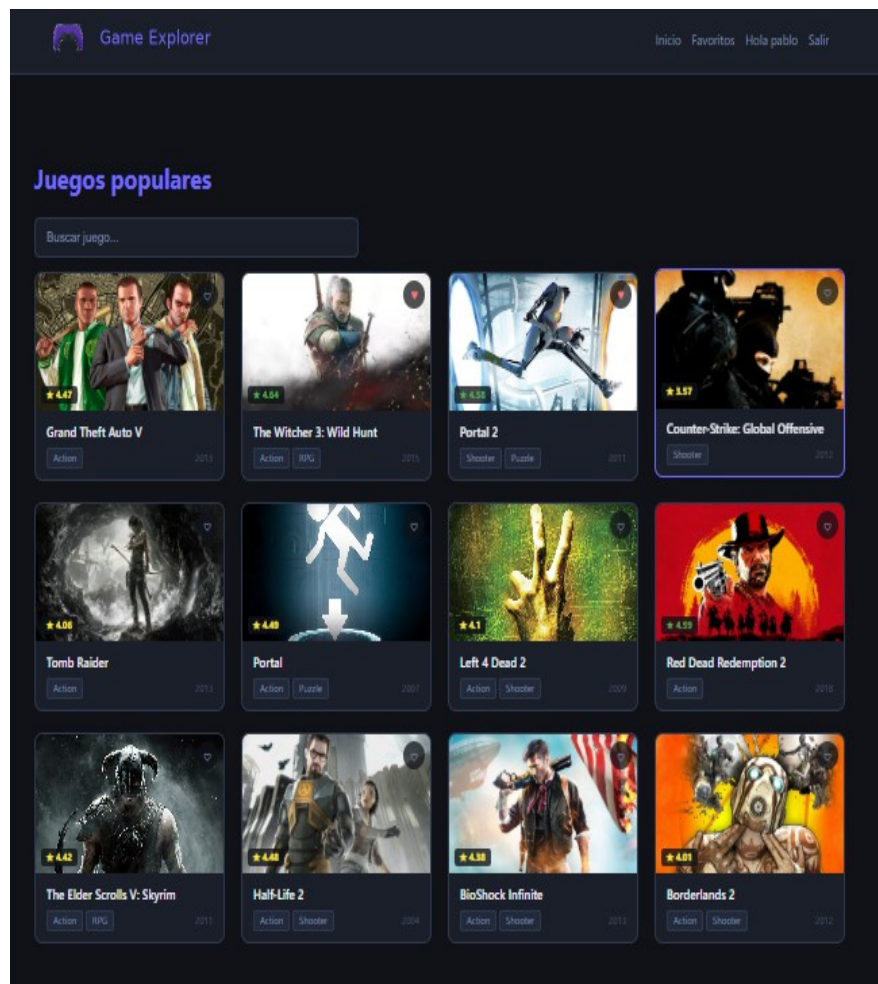
5. Organización de componentes

- **Views:** representan las páginas principales de la aplicación (Inicio, Detalle, Favoritos, Login).
- **Components:** contienen elementos reutilizables de la interfaz, como tarjetas de juegos, barra de navegación y pie de página.
- **Stores:** se utiliza Pinia para centralizar la gestión de los favoritos, permitiendo que distintas vistas compartan esa información.
- **Composables:** encapsulan lógica reutilizable para evitar duplicar código y mantener los componentes enfocados en la presentación.
- **Services:** concentran las funciones encargadas de consumir la API externa.

6. Instrucciones de ejecución

1. Clonar el repositorio.
2. Ejecutar `npm install` para instalar las dependencias.
3. Ejecutar `npm run dev` para iniciar el servidor de desarrollo.
4. Abrir la dirección indicada por Vite en el navegador.

7. Capturas



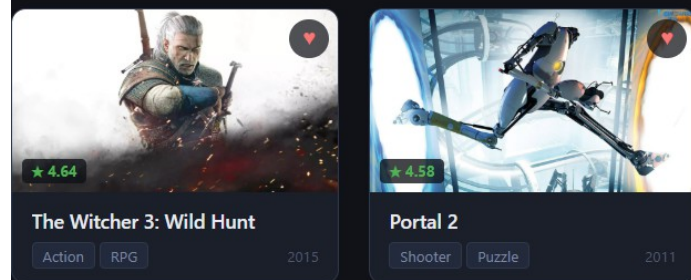
Detalles del juego:

Fecha de lanzamiento: 2013-09-17

Puntuación: ★ 4.47 / 5

Sitio web: <http://www.rockstargames.com/V/>

Mis Favoritos



Iniciar sesión

Usuario

pablo

Contraseña

1234

Entrar